

Labo 02 — Questions

Répondez sans relire la théorie. Justifiez toujours : une affirmation sans mécanisme ne vaut rien.

Question 1 [Compréhension]

Écrivez le nom **complet et explicite** que le moteur construit à partir de chacune de ces écritures, expliquez la règle qui permet de savoir si la première partie est un registry ou un namespace — et dites pour chacune comment Podman se comporte face au nom court :

```
nginx
bitnami/nginx
registry.masociete.be:5000/socle/nginx:1.25
```

Question 2 [Analyse]

Deux serveurs, A et B, ont tiré `monapp/api:2.3` le même jour. Trois semaines plus tard, l'équipe redéploie sur B uniquement, avec la même commande `podman pull monapp/api:2.3`. Un bug apparaît sur B et pas sur A. Comment cela est-il possible alors que la version est « la même » ? Quelle commande permet de le prouver, et quelle pratique aurait évité le problème ?

Question 3 [Diagnostic]

Un développeur observe que `podman images` liste 40 images pour un total de 62 Go dans la colonne `SIZE`, alors que son disque WSL ne fait que 100 Go et qu'il n'a jamais eu de problème de place. A-t-il vraiment 62 Go d'images ? Expliquez, donnez la commande qui affiche le chiffre réel, et dites où ces fichiers se trouvent physiquement sur son poste.

Question 4 [Analyse]

Un Dockerfile contient :

```
COPY credentials.json /tmp/credentials.json
RUN ./configure.sh && rm /tmp/credentials.json
```

L'auteur affirme que le secret n'est pas dans l'image finale puisqu'il l'a supprimé. A-t-il raison ? Expliquez précisément ce que contient l'image, et pourquoi le `rm` ne change rien au problème.

Question 5 [Compréhension]

Vous poussez une deuxième version de votre image Spring Boot vers le registry. Elle pèse 310 Mo, la précédente pesait 308 Mo. Le `push` ne transfère pourtant que 61 Mo. Expliquez le mécanisme, puis dites ce que vous devriez changer dans votre Dockerfile si le push transférait à chaque fois les 310 Mo.

Question 6 [Diagnostic]

```
$ podman images
REPOSITORY          TAG          IMAGE ID      SIZE
localhost/api       2.0         f3a1b9c02d11  310 MB
localhost/api       1.9         f3a1b9c02d11  310 MB
<none>              <none>      8b2c74e91a03  295 MB
```

Commentez cette sortie : combien d'images distinctes voyez-vous réellement ? D'où vient la ligne `<none>` ? Que se passe-t-il exactement si vous tapez `podman rmi api:1.9` ? Et pourquoi `localhost/` ?

Question 7 [Analyse]

Votre collègue construit l'image du back sur son MacBook M3 et la pousse au registry. Le déploiement sur le serveur de recette échoue avec `exec /usr/bin/java: exec format error`. Diagnostiquez, et donnez deux façons de corriger — l'une pour dépanner tout de suite, l'autre pour que le problème ne se reproduise plus.

Question 8 [Compréhension]

`podman save` et `podman export` produisent tous deux une archive `.tar`. Dans quelle situation chacun est-il le bon choix ? Que perd-on précisément si l'on utilise `export` pour transporter une image Spring Boot vers un site isolé ? Et que change `--format oci-archive` sur un `save` ?

Question 9 [Analyse]

Après un `podman pull` d'une image de 400 Mo, vous relancez la même commande : elle se termine en moins d'une seconde. Que le moteur a-t-il réellement vérifié — et pourquoi cela n'a-t-il presque rien coûté en réseau ?

Question 10 [Diagnostic]

La CI de votre entreprise échoue par intermittence sur `podman pull node:22-alpine`, avec le message `toomanyrequests: You have reached your pull rate limit`. Aucun changement n'a été fait dans le pipeline. Expliquez la cause, pourquoi elle apparaît « par intermittence », et les deux réponses classiques en entreprise.

Question 11 [Diagnostic]

Vous lancez un registry local (`podman run -d -p 5000:5000 registry:2`), puis :

```
$ podman push localhost:5000/socle/demo:1.0
Error: ... pinging container registry localhost:5000: Get "https://localhost:5000/v2/":
http: server gave HTTP response to HTTPS client
```

Votre collègue, sous Docker, n'a jamais vu ce message avec le même registry. Expliquez la différence de philosophie, donnez deux façons de faire passer le `push` — et dites pourquoi l'une des deux ne doit jamais atteindre un fichier de configuration versionné.

Question 12 [Diagnostic]

Un `podman rmi mon-api:1.0` renvoie :

```
Error: image used by 4c2e9a1b7d33...: image is in use by a container: consider listing
external containers and force-removing image
```

Expliquez la situation, dites pourquoi le moteur refuse, donnez la manière **propre** de résoudre — puis dites ce que fait `podman rmi -f` et pourquoi c'est une mauvaise idée ici.

Question 13 [Analyse]

`podman history` sur une image montre plusieurs couches de taille `0B` et une couche de `180MB`. Que sont les couches à `0B`, et pourquoi existent-elles quand même ? En quoi cette lecture vous

aide-t-elle concrètement à réduire la taille d'une image ?

Question 14 [Compréhension]

Votre équipe hésite entre trois stratégies de tags pour l'image du back : (a) `api:latest` réécrit à chaque build, (b) `api:1.4.2` suivant la version applicative, (c) `api:1.4.2-b318-a9f3c21` incluant numéro de build et *commit* Git. Discutez les trois du point de vue du **retour arrière en production** et du **diagnostic d'incident**.